ChronoForge Pulse

Slides Explained / Presentation 2 - EDA & Data Preprocessing

What this document is

A single companion PDF for the 18-slide deck. Every slide gets one page that follows the same structure: a one-line tagline, a short **Summary**, the **Key points** as bullets, and a **How to present it** paragraph for the speaker. After the 18 slide pages, the document closes with an **Extra Knowledge** section covering the business rationale for lot-sizing, the data workflow, the feature deep-dive, how the features feed each lot-sizing algorithm, and the future direction beyond this deck.

The story in one paragraph

ChronoForge Pulse is an AI-native decision platform for supply-chain planning. We use eight years of monthly telecom-subscriber data from Bangladesh (six operators, Jan-2008 to Dec-2015) as a stand-in for any demand signal a supply-chain team has to plan around. This deck - Presentation 2 - is the data-preparation stage: reshape the raw workbook into a tidy table (576 rows x 7 columns), engineer two demand-meaningful features (**MoM_Growth_%** and **Market_Share_%**), run quality / statistical / outlier checks, and export the clean dataset. The result feeds Presentation 3, where an ensemble of **ARIMA + LSTM + XGBoost** produces the forecast and three lot-sizing algorithms (**EOQ, Wagner-Whitin, Silver-Meal**) convert that forecast into actual order quantities.

Reading order

- Pages 2 - 19: one page per slide, in deck order.
- Pages 20+: Extra Knowledge - the business and technical background that did not fit on the slides themselves.

SLIDE 01 / 18

Title / Cover

Who we are and what we are presenting.

Summary

The cover slide names the project (ChronoForge Pulse), the deck (EDA & Data Preprocessing), the dataset (Telecom Market in Bangladesh, 2008-2015), the engine (ARIMA / LSTM / XGBoost + 3 lot-sizing algorithms), the four group members and the course.

Key points

- This is Presentation 2 of 3 in the ChronoForge Pulse project.
- Eight years of monthly subscriber data, six operators.
- The end goal is forecasts + buy plans, not just charts.
- Team: Kawser, Asir, Labib, Mujahidul (DBI major).

How to present it

Open with: *Today is the data-prep stage of our AI-native supply-chain project. We are taking a real, eight-year telecom dataset and turning it into something an ensemble forecaster can learn from.* Then point at the engine line - that is where the entire project is heading.

SLIDE 02 / 18

What is ChronoForge Pulse?

An AI-native engine for forecasting & lot-sizing.

Summary

One-line definition plus three model-family cards (ARIMA, LSTM, XGBoost) and a fourth card for lot-sizing. The footer strip we added makes the I/O explicit: INPUTS are demand signals, OUTPUTS are forecasts plus buy plans.

Key points

- ARIMA - classical statistics, captures trend + seasonality.
- LSTM - deep learning, captures long-range temporal patterns.
- XGBoost - gradient-boosted trees, captures non-linear feature interactions.
- Lot-sizing turns the forecast into *order quantities* - without it, the forecast is just a chart.
- Three families instead of one because no single model class covers all the regimes (trend, volatility, seasonality, feature interactions).

How to present it

Frame the three model families as *three different lenses on the same demand series*. ARIMA is the statistician, LSTM is the memory, XGBoost is the interaction-finder. The lot-sizing block at the right is the procurement layer that translates the forecast into action. Mention that the deck today is what makes all of this possible - clean data, right shape, the right features.

SLIDE 03 / 18

Project Roadmap

Three presentations, one platform.

Summary

Three numbered cards laid out left-to-right: Presentation 1 (problem framing & business context), Presentation 2 (today - EDA & data preprocessing), Presentation 3 (ML modelling & lot-sizing).

Key points

- Presentation 1 defined the platform and the use case.
- Presentation 2 (today) turns raw data into a model-ready dataset.
- Presentation 3 will train ARIMA / LSTM / XGBoost and run the three lot-sizing algorithms.
- Each deck depends on the previous one - data quality from today directly shapes the modelling accuracy tomorrow.

How to present it

Point at card 2 and say: *This is where we are.* Then point at card 3 and say: *And this is the payoff - forecasts and order plans. Without today's work, the work in card 3 is unreliable.*

SLIDE 04 / 18

Scope of This Presentation

What we do today vs. what comes next.

Summary

Two parallel lists. Left (**In Scope**): load raw CSV, reshape wide->long, type casting, missing / duplicate checks, feature engineering, descriptive stats, visual EDA, outlier detection, export. Right (**Next - Presentation 3**): train ARIMA / LSTM / XGBoost, ensemble blend, evaluate, run three lot-sizing algorithms, deploy.

Key points

- Left = today; right = next deck.
- Every item in the left column maps to a slide later in this deck.
- The export step at the bottom of the left column is the deliverable: telecom_clean.csv.
- Lot-sizing on the right uses EOQ, Wagner-Whitin and Silver-Meal - three different algorithms for three different demand shapes.

How to present it

Walk down the left list, naming each step quickly. Then run down the right list - just enough to show how today's exported CSV becomes tomorrow's training data. End on the phrase *model-ready dataset* - that is what we are producing today.

SLIDE 05 / 18

Pipeline - Inputs & Outputs

Wide & messy in. Tidy & tabular out.

Summary

The architecture diagram of the data pipeline. Three panels: INPUT (78x112 raw CSV with multi-row header), EDA + PREPROCESSING (10 numbered steps), OUTPUT (576x7 tidy CSV with 0 nulls, 0 duplicates, 51 outliers flagged). The footer note explains the multi-row header detail.

Key points

- INPUT: 78 x 112, wide, mixed with assumption rows.
- 10 preprocessing steps - each one is a slide later in the deck.
- OUTPUT: 576 x 7 tidy rows, ready for ARIMA / LSTM / XGB.
- Granularity stays monthly - we never aggregate up or down.
- Quality numbers at the right (0 nulls, 0 dupes, 51 outliers) are the proof the pipeline worked.

How to present it

Start at INPUT and move left-to-right. *Wide, messy, human-readable - then ten transformations - then a long, tidy, machine-trainable table.* The three boxes are the elevator pitch of the whole deck.

SLIDE 06 / 18

Dataset Overview

Eight years of subscriber demand.

Summary

Source (Grameenphone internal market model / BTRC data), period (Jan-2008 to Dec-2015, 96 months), six operators, units (millions), raw vs. cleaned shape. The bottom strip we added shows sector growth 506 -> 1,536 Mn-months and the proxy callout (SIM cards, handsets, recharge, tower equipment).

Key points

- Six operators: Grameenphone, Banglalink, Robi, Airtel, Citycell, Teletalk.
- 96 monthly observations per operator - enough for both ARIMA and LSTM.
- Subscriber count is a strong proxy for downstream physical demand.
- Sector grew ~3x in eight years - strong upward trend.
- Source is industry-standard (Grameenphone market model based on BTRC regulator data).

How to present it

Audiences trust an analysis only if they trust the data. Walk through source, period, operators, units. Then make the proxy argument: *If subscribers are growing, SIMs, handsets and tower equipment all have to grow with them - this is exactly the demand a supply chain has to plan around.*

SLIDE 07 / 18

Raw Data Structure

A spreadsheet, not a table.

Summary

A simplified preview of the raw layout: operators down the rows, months across the columns (Jan-08 ... Dec-15). Three bottom cards we added explain WHY WIDE (planning workbook), WHAT WE STRIP (assumption / sector-total rows), and TARGET SHAPE (one row per Operator x Month).

Key points

- Raw layout: 78 rows x 112 columns - human-readable.
- Each operator's full eight-year history sits in one row.
- Trailing Avg-12M / assumption columns are derived, not observed - dropped.
- Sector-total row is the sum of the operators - also dropped.
- Target: one row per *observation*, not per *operator*.

How to present it

Point at the wide preview and ask the audience: *could you train a time-series model on this shape?* Answer: no - models need one observation per row. So the next slide does the reshape.

Reshaping - Wide -> Long (Tidy)

Unpivot to tidy.

Summary

Before / After tables side by side, with the pandas .melt() arrow between them. The pipeline code block we added shows the actual chain: melt → to_datetime → dt.year / dt.month_name. Result: 6 operators x 96 months = 576 tidy rows.

Key points

- .melt() stacks every monthly column into one long column called Subscribers_Mn.
- Period strings (e.g. Jan-08) are parsed into real datetime objects.
- Year (int) and Month name (str) are extracted for use as calendar features.
- No data lost - the row count is exactly 6 x 96 = 576.
- The shape change is what unlocks groupby, lag features, and per-series model training downstream.

How to present it

This is the single most important transformation of the deck. Walk through the pandas chain line by line: melt reshapes, to_datetime parses, dt.year / dt.month_name extracts. End on the result line: *6 operators x 96 months = 576 rows - same data, machine-ready shape.*

SLIDE 09 / 18

Data Quality Checks

Clean by construction.

Summary

Four big numbers: 576 rows, 7 columns, 0 nulls, 0 duplicates. Plus checks: groupby-interpolate -> ffill -> bfill safety net on Subscribers_Mn; datetime / float types verified; numerical vs. categorical columns listed. Our footer note adds the row-math: 576 = 6 operators x 96 months.

Key points

- **0 nulls** per column after tidying - the dataset is fully dense.
- **0 duplicates** on the (Operator, Date) key - every observation is unique.
- Safety-net interpolation runs per operator, so a gap is filled by that operator's own neighbours - never by a different operator.
- Types verified: Date is datetime64, Subscribers_Mn is float64 - otherwise time operations would silently fail.
- Categorical vs. numerical column list is the input contract that Presentation 3 reads.

How to present it

The 0/0 headline is the trust statement. Then explain the safety net briefly - *even if a value were missing, it would have been filled using only that operator's own history.* Land on: *the dataset is now a clean contract for the modelling layer.*

Feature Engineering

Two features that carry the signal.

Summary

The two engineered features: **MoM_Growth_%** (percent change of Subscribers_Mn within each Operator) and **Market_Share_%** (Subscribers_Mn divided by the Date-level sector total, x 100). Plus a CALENDAR row for Year and Month name as seasonality / trend regressors. The two spec cards we added show the actual pandas calls.

Key points

- **MoM_Growth_%** = df.groupby('Operator')['Subscribers_Mn'].pct_change() x 100 - normalised growth.
- **Market_Share_%** = subs / df.groupby('Date')['Subscribers_Mn'].transform('sum') x 100 - competitive lens.
- Growth strips out scale - a 0.5 Mn jump means different things at Teletalk vs. Grameenphone; the percentage normalises that.
- Share & level are correlated at 0.88 but not identical - share is essentially demand-mix.
- **Calendar** features become seasonality dummies and trend regressors for XGBoost and ARIMA.

How to present it

Tell the audience: *raw subscriber counts are not enough. We need a growth signal and a competition signal.* Then walk through the two cards - one for MoM_Growth_%, one for Market_Share_%. Close on the calendar row: *plus Year and Month name for seasonality.*

Statistical Summary

A market with one giant.

Summary

Per-operator subscriber statistics in millions (Min, Mean, Max, Std). Three insights: GP leads on every measure; top three have high std (= growth, not noise); Citycell is flat & tiny. Our GAP callout: GP mean (36.03 Mn) is about 22x Citycell (1.61 Mn).

Key points

- **Grameenphone**: min 16.88, mean 36.03, max 56.54, std 12.16 - leader on every measure.
- **Banglalink, Robi**: strong followers, double-digit means.
- **Airtel**: mid-volume, mean ~5.7 Mn.
- **Teletalk, Citycell**: long tail, means under 2 Mn.
- GP / Citycell mean ratio ~ 22x - the market structure is extremely asymmetric.

How to present it

Numbers first, then the story. *The mean column tells you who is big; the std column tells you who is moving. GP is both - the leader is still growing fast.*

SLIDE 12 / 18

Subscriber Growth Over Time

The whole market is climbing.

Summary

Monthly subscriber line chart for all six operators, 2008-2015. Side notes: three clear winners (GP, Banglalink, Robi); Citycell is the exception (rises then declines); wide dynamic range from sub-1 Mn to 56 Mn.

Key points

- GP is the top line - climbing from ~17 Mn to ~57 Mn over eight years.
- Banglalink and Robi follow with similar upward slopes.
- Airtel is a slower but steady climb.
- Teletalk is small but trending up.
- Citycell is the only line that flattens and then declines.
- The huge spread on the y-axis is itself an argument for an ensemble model.

How to present it

Numbers showed it; the chart proves it. Point at each line and name the operator. Land on Citycell - *the only operator whose forecast should point downward.*

SLIDE 13 / 18

Subscriber Distribution by Operator

Spread tells the growth story.

Summary

Boxplot of subscribers per operator. Side notes: tall boxes = big movers (GP spans 17-57 Mn); tight boxes = stable players (Citycell sits in a thin band near 1.6 Mn); the box approximates mean +/- 0.67 std.

Key points

- GP's box is huge - eight years of growth made visible.
- Citycell's box is a thin band - basically no variation.
- Box height ranks operators by *how much they moved*, not by their average size.
- The median line inside each box approximates that operator's mean.
- This shape directly motivates per-operator forecast strategy.

How to present it

Compare GP's tall box to Citycell's thin one. *One was a story of growth; the other was a story of stagnation.* Mention that the box approximation rule (mean +/- 0.67 std) lets you eyeball each operator's volatility instantly.

SLIDE 14 / 18

Average Market Share

Three operators own 89%.

Summary

Bar chart of average market share per operator: GP 43.3%, Banglalink 25.5%, Robi 20.0%, Airtel 6.6%, Citycell 2.3%, Teletalk 2.2%. Side notes summarise: a clear leader, the top three at ~89%, the long tail at ~11%.

Key points

- GP at 43.3% holds nearly half the market alone.
- Top three (GP + Banglalink + Robi) take ~89% of demand.
- Long tail (Airtel + Citycell + Teletalk) is barely 11%.
- This concentration directly shapes the lot-sizing strategy: big stable batches for the top three; lumpy small batches for the tail.

How to present it

Tell the supply-chain story: *if you are buying for the sector, your buying budget is dominated by three operators. EOQ works well for them. The small tail needs Wagner-Whitin or Silver-Meal because their demand is too lumpy for EOQ assumptions.*

Correlation & Sector-Wide Growth

Share is demand. Demand is rising.

Summary

Two charts. Left: correlation matrix between Subscribers_Mn, MoM_Growth_% and Market_Share_% (Subs <-> Share = 0.88; MoM near-independent of level). Right: yearly sector total grew 506 -> 1,536 Mn-months.

Key points

- Subscribers <-> Market_Share correlation = 0.88 - forecasting share is essentially forecasting demand mix.
- MoM growth is near-independent of the level (corr 0.01) - good: it is a distinct signal.
- Yearly totals: 506 (2008) -> 1,536 (2015) - ~3x in eight years.
- Strong upward trend means ARIMA's trend term and LSTM's linear projection will both work hard.

How to present it

Two checks at once: *(a) our features behave the way we expected and (b) the underlying market is genuinely growing.* 0.88 is a high but not perfect correlation - share and level carry overlapping but not identical signal.

Outlier Detection (IQR)

51 outliers - real shocks, not errors.

Summary

IQR method applied to MoM_Growth_%. Bounds: -3.07% to +5.50%. 51 monthly observations fall outside that band, concentrated in the early-period growth spurts of Airtel, Banglalink and Robi. The CONSEQUENCE line we added: shocks are not noise - LSTM + XGBoost must model the volatility and lot-sizing must hold safety stock.

Key points

- IQR rule: anything below Q1 - 1.5*IQR or above Q3 + 1.5*IQR is an outlier.
- Lower bound -3.07% / upper bound +5.50% - tight bounds because most months are smooth growth.
- 51 violations concentrated in early years of fast-growing operators.
- We **do not remove** them - they are real demand events (launches, campaigns, expansions).
- Flagging them lets the ensemble learn the volatility regime explicitly.

How to present it

Make the distinction clear: *in finance you might delete outliers; in demand forecasting they are exactly the events the supply chain has to plan for.* The 51 flagged months drive the safety-stock layer in the lot-sizing plan.

What the EDA Lets Us Predict

From patterns to predictions.

Summary

Six cards summarising the hypotheses Presentation 3 will quantify: SECTOR TREND (~3x), OPERATOR SHAPE (three rising, one declining, one slow), VOLATILITY (shocks are recurring), MARKET-SHARE SIGNAL (share = demand-mix), LOT-SIZING (big vs. lumpy), LOW-VOLUME (Teletalk slow but positive).

Key points

- Sector demand is on a strong upward trend (3x).
- Per-operator shape splits into three patterns: monotonic growth, slow-steady, and decline.
- Volatility is a real and recurring feature, not noise.
- Share ~ demand correlation gives a second forecast lens.
- Lot-sizing assignment: EOQ for stable giants; Wagner-Whitin / Silver-Meal for lumpy small operators.

How to present it

This is the bridge slide. *The EDA has told us six things; Presentation 3 has to quantify each of them.* Read each card title and pair it with one number from the deck: ~3x, 89%, 0.88, 51, 22x.

SLIDE 18 / 18

Thank You / Deck Highlights

Clean data. Ready to forecast.

Summary

Closing slide with the tagline and the HIGHLIGHTS panel we added: 576 x 7 tidy shape, 0/0 nulls and duplicates, 2 engineered features, 51 IQR-flagged shocks, 3x sector growth - five at-a-glance numbers.

Key points

- **576 x 7** - tidy shape delivered to the ensemble.
- **0 / 0** - nulls / duplicates after safety net.
- **2** - engineered features (MoM_Growth_%, Market_Share_%).
- **51** - IQR outliers flagged, not deleted.
- **3x** - sector growth 2008 -> 2015 (506 -> 1,536 Mn-months).

How to present it

Audiences remember the last number they saw. Read all five out loud, then end with: *this is the clean foundation that feeds the ARIMA + LSTM + XGBoost ensemble and the three lot-sizing algorithms in Presentation 3.* Invite questions.

EXTRA KNOWLEDGE

Six short essays that sit behind the deck. They cover the **business rationale** for the project, the **data workflow**, the **feature deep-dive**, how the features **feed the lot-sizing plan**, the **lot-sizing parameters (D, S, H) for this dataset** with EOQ / Wagner-Whitin / Silver-Meal worked examples, and the **future direction** beyond Presentation 3.

1. Why we use a lot-sizing plan

A forecast tells you what is coming. A lot-sizing plan tells you what to buy and when.

The forecast alone is not enough

A forecast says *next month we expect demand of X units*. What it does not say is *so when do we place an order, and how big should that order be?* A demand chart on its own does not move a single unit of inventory. Lot-sizing is the bridge between demand forecasts and procurement decisions - it is the layer that converts a forecast into action.

The three costs we are trading off

- **Ordering cost** - every time you place an order there is a fixed cost: paperwork, supplier setup, transport mobilisation, customs, internal approvals. More orders = more ordering cost.
- **Holding cost** - every unit sitting in inventory costs money: warehouse space, capital tied up, shrinkage, obsolescence. Bigger stock = more holding cost.
- **Stock-out cost** - if a surge month arrives and you do not have stock, you lose revenue, customer trust, and potentially market share to a competitor.

Lot-sizing is the formal solution to this three-way trade-off. Order too often and ordering cost is heavy; order too rarely and holding cost (or stock-out risk) is heavy. The algorithm picks timing and quantity that minimise the total cost.

Why three different algorithms

- **EOQ (Economic Order Quantity)** - assumes constant demand. Formula: Q* = sqrt(2 D S / H) where D is annual demand, S is order cost, H is holding cost per unit per period. Right fit for the stable giants (GP, Banglalink, Robi).
- **Wagner-Whitin** - dynamic programming over the full demand vector. Cost-optimal for time-varying, lumpy demand. We use it for the small / volatile tail (Citycell, Teletalk) where the EOQ assumption of constant demand breaks.
- **Silver-Meal** - greedy heuristic: keep extending the current order until average cost per period stops falling. Practical middle ground - good for Airtel-shaped series where Wagner-Whitin is overkill but EOQ is too rigid.

The business impact

In a telecom supply chain - SIM cards, handsets, recharge stock, tower equipment - inventory decisions move millions of taka per month. A 5% reduction in over-ordering pays back the entire modelling effort in a single quarter. And a well-tuned safety stock means that the 51 surge months we flagged in slide 16 do not become stock-out events.

2. How we work with the data

Seven steps from raw spreadsheet to model-ready dataset.

Step 1 - Load the raw CSV

We load GP Model.xlsx - Telecom Mkt. in Bangladesh.csv with header=None. The reason for header=None is that the source workbook has a *multi-row header* - a Year row stacked above a Month row - so pandas cannot infer the header automatically. Shape after load: 78 rows x 112 columns.

Step 2 - Slice the real observations

Rows 4 to 10 contain the six operators. Columns 2 to 97 hold the 96 monthly subscriber values (Jan-2008 through Dec-2015). We drop the sector-total row (it is derived, not observed), the Avg-12M / assumption columns (they are model inputs, not data), and the blank padding.

Step 3 - Reshape wide to long

pandas.melt() unpivots the operator x month matrix into a tidy long format with one row per (Operator, Month). We then parse the period string into a real datetime and extract Year and Month name. Final shape: 576 rows (6 operators x 96 months) x 7 columns.

Step 4 - Quality gate

- Missing values per column - all zero after tidying.
- Duplicate (Operator, Date) keys - zero.
- Safety-net interpolation: df.groupby('Operator').interpolate() -> ffill() -> bfill() on Subscribers_Mn - so any gap is filled from the operator's own history.
- Type verification: Date as datetime64, Subscribers_Mn as float64.

Step 5 - Engineer features

Two demand features (MoM_Growth_%, Market_Share_%) and two calendar features (Year, Month name). See section 3 for the deep-dive.

Step 6 - Statistics & outliers

Per-operator min / mean / max / std and an IQR filter on MoM_Growth_%. Bounds are -3.07% to +5.50%; 51 months fall outside. We **flag** them - we do not delete - because they are real demand events the supply chain has to plan for.

Step 7 - Export

Final artefact: telecom_clean.csv with columns Operator, Date, Year, Month, Subscribers_Mn, MoM_Growth_%, Market_Share_%. This is the exact file Presentation 3 reads to train ARIMA / LSTM / XGBoost and to run the three lot-sizing algorithms.

3. Feature deep-dive

What each engineered feature is, how it is computed, and what signal it carries.

MoM_Growth_%

df.groupby('Operator')['Subscribers_Mn'].pct_change() x 100. The percent change in subscribers from one month to the next, computed *within* each operator's own series.

- **Why grouped** - so the percentage starts fresh for each operator rather than crossing series boundaries.
- **What signal it carries** - velocity. Tells the models how fast each operator is moving right now.
- **Why percent, not absolute** - a 0.5 Mn jump means different things at Teletalk vs. Grameenphone. The percentage strips out absolute scale.
- **Where it is used downstream** - XGBoost lag features, LSTM input window, the IQR outlier detector.

Market_Share_%

subs / df.groupby('Date')['Subscribers_Mn'].transform('sum') x 100. The fraction of the sector each operator owns at every Date, expressed as a percentage.

- **Why transform('sum')** - to broadcast the per-Date total back onto every row so the division is element-wise.
- **What signal it carries** - competitive position. Tells the models who is gaining and who is losing share at every point in time.
- **Why useful even with high correlation to level** - the correlation is 0.88, not 1.0; share captures relative movement that level alone misses.
- **Where it is used downstream** - XGBoost feature input; drives the demand-mix forecast that lot-sizing uses to allocate orders across operators.

Calendar features (Year, Month)

df['Year'] = df['Date'].dt.year and df['Month'] = df['Date'].dt.month_name(). The integer year and the human-readable month name.

- **Year** - acts as a linear trend regressor for ARIMA and XGBoost; lets the model project the 3x sector growth into the future.
- **Month name** - one-hot encoded into seasonality dummies; captures month-specific patterns (festivals, school years, fiscal cycles).

IQR outlier flag

A binary column derived from the IQR rule on MoM_Growth_%. 51 months are flagged. This is not a model feature in the strict sense - it is a **safety-stock driver**: the count of historical flags per operator becomes the volatility input to the safety-stock formula in the lot-sizing layer.

4. How features feed the lot-sizing plan

Features feed the forecast; the forecast feeds the lot-sizer.

The indirection in one sentence

Lot-sizing algorithms do not consume raw history. They consume a **future demand vector** (next N periods of expected demand) plus a small set of cost parameters. So the EDA features feed the lot-sizing plan *indirectly*: they enter the forecasting models, the models output the demand vector, and the vector is what the lot-sizing layer optimises against.

Feature -> model -> lot-sizing role

Feature	Used by (forecast)	Role in the lot-sizing plan
Subscribers_Mn	ARIMA, LSTM, XGBoost	level series. The forecast of this number becomes the demand vector that
MoM_Growth_%	LSTM, XGBoost	Velocity. In high-growth months the lot-sizer responds with a larger order quantity.
Market_Share_%	XGBoost	Demand-mix lens. Lets the lot-sizer shift order weight between operators when sh
Year	ARIMA, XGBoost	Trend regressor. Extrapolates the 3x sector growth into the planning horizon.
Month name	ARIMA, XGBoost	Seasonality. Lets the lot-sizer anticipate month-specific peaks (festival months, fis
IQR outlier flag	Safety-stock formula	Volatility input. More historical flags -> bigger safety buffer added on top of the

How each algorithm uses the demand vector

- **EOQ** takes the *average* demand over the horizon and computes one fixed order quantity. Great fit for high-volume, smooth demand series - GP, Banglalink, Robi.
- **Wagner-Whitin** takes the full month-by-month forecast vector and runs dynamic programming to find the cost-minimising schedule of timings and quantities. Optimal but more compute. Best for lumpy / declining series - Citycell, Teletalk.
- **Silver-Meal** takes the same vector and uses a greedy heuristic: extend the current order as long as average cost per period keeps falling. Practical for mid-volume series like Airtel.

The safety-stock layer

On top of whatever the lot-sizer recommends, we add a safety stock buffer for the 51 IQR-flagged months. A common form is SS = z * sigma_demand * sqrt(lead_time). Higher historical volatility -> bigger sigma -> bigger buffer. So the outlier flag we engineered in EDA is what protects the supply chain when a surge month arrives.

5. Lot-sizing parameters in this dataset

What D, S and H actually are for the telecom series, and how each of the three algorithms uses them.

The mapping problem first

Subscribers themselves are not an inventory item - you cannot stock a subscriber. So we treat the subscriber series as a **demand proxy** for a physical SKU the supply chain actually buys. The four realistic candidates are SIM cards, handsets, recharge / scratch cards, and tower / network equipment. The cleanest mapping is **SIM cards** (one SIM per new subscriber), so the examples below use SIMs for Grameenphone.

D - Annual demand (in the dataset)

D is the only one of the three parameters that comes directly from telecom_clean.csv. For SIM cards, D is the annual *net* new-subscriber count per operator: D = (Subscribers_Mn[end] - Subscribers_Mn[start]) x 1,000,000 / years. (Gross adds would also need churn, which we do not have.)

Operator	2008 (Mn)	2015 (Mn)	Delta (Mn)	D (units/yr)
Grameenphone	16.88	56.54	39.66	~ 4.96 M
Banglalink	7.45	33.39	25.94	~ 3.24 M
Robi	7.28	28.94	21.66	~ 2.71 M
Airtel	2.37	9.82	7.45	~ 0.93 M
Teletalk	1.00	4.28	3.28	~ 0.41 M
Citycell	1.45	1.05	-0.40	~ 0 (decline)

For EOQ you collapse the series into a single D. For Wagner-Whitin and Silver-Meal you keep the **full monthly demand vector** produced by the ARIMA / LSTM / XGBoost forecast.

S - Setup / ordering cost (not in the dataset)

S is a procurement parameter that has to be supplied externally - paperwork, supplier setup, freight booking, QA, customs clearance. Industry-realistic ranges for a BD telecom SIM order are **$500 - $1,500 per order** for a small / domestic supplier and **$2,000 - $5,000** for a large imported batch. The worked examples below use **S = $1,000 per order**.

H - Holding cost per unit per year (not in the dataset)

H combines capital cost, warehousing, shrinkage and obsolescence. A common rule of thumb is 15 - 25 % of unit cost per year. For a SIM card at $0.30, H ≈ 0.20 x $0.30 = **$0.06 per SIM per year**.

EOQ worked example - Grameenphone SIMs

EOQ assumes constant demand. Plug D, S, H into the closed-form minimiser of total ordering + holding cost:

Q* = sqrt(2 . D . S / H)

Q* = sqrt(2 . 4,960,000 . 1,000 / 0.06) ≈ 406,500 SIMs per order

Metric	Value
Optimal order quantity Q*	~ 406,500 SIMs
Orders per year (D / Q*)	~ 12 (monthly)
Days of supply per order	~ 30 days
Average inventory (Q* / 2)	~ 203,250 SIMs
Annual ordering cost (D/Q*) . S	~ $12,200
Annual holding cost (Q*/2) . H	~ $12,200
Total annual cost (ordering + holding)	~ $24,400

At the optimum, ordering cost and holding cost are equal - that is the defining property of EOQ. Any deviation from Q* raises one of them faster than it lowers the other.

Wagner-Whitin - the cost-optimal dynamic plan

Wagner-Whitin (1958) is a dynamic-programming algorithm that produces the **cost-minimising** production / order plan for a finite horizon of time-varying demand. Unlike EOQ it does not assume constant demand - it consumes the full forecast vector d_1, d_2, ..., d_T.

How the algorithm works

- **State**: f(t) = minimum cumulative cost to satisfy demand from period 1 through period t.
- **Decision**: at each period t, decide which earlier period j (j ≤ t) the order that covers t was placed in.
- **Recurrence**: f(t) = min over j≤t [f(j-1) + S + sum_{k=j..t} H . (k - j) . d_k]
- **Property exploited**: in an optimal plan, an order is *only* placed when the previous order's stock has run to zero. So we never need to consider partial coverage - this is what makes the DP tractable.
- **Complexity**: O(T^2) for the basic version, O(T log T) with the Federgruen-Tzur speedup.

Worked example - Citycell SIMs over 6 months

Suppose the forecast for Citycell over the next 6 months is d = [20, 40, 30, 10, 50, 25] (in thousands), with S = $1,000 and H = $0.06/unit/yr (= $0.005/unit/month).

t	d_t (k)	f(t) ($)	Order placed in period
1	20	1,000	1 (covers 1)
2	40	1,200	1 (covers 1-2, 20+40 batched)

3	30	2,200	3 (new order, covers 3)
4	10	2,250	3 (covers 3-4)
5	50	3,250	5 (new order, covers 5)
6	25	3,375	5 (covers 5-6)

Optimal plan: **three orders**, in months 1, 3 and 5, covering (1-2), (3-4), (5-6) respectively. Total cost f(6) ≈ $3,375 - lower than any fixed Q* EOQ would produce on this lumpy series. Wagner-Whitin is our default for Citycell and Teletalk.

Silver-Meal - the fast greedy heuristic

Silver-Meal (1973) is a simple heuristic that produces near-optimal plans with O(T) work. The rule: *keep extending the current order's coverage as long as the average cost per period keeps falling; stop the moment it rises.*

The decision rule

- Start a new order in period j. Let n be how many periods this order will cover (n = 1, 2, 3 ...).
- **Average cost per period** if this order covers n periods: AC(n) = (S + H . sum_{k=1..n-1} k . d_{j+k}) / n.
- **Extend** coverage by one more period while AC(n+1) < AC(n); **stop** as soon as AC(n+1) >= AC(n). Then start a fresh order at period j + n.

Worked example - Airtel SIMs

Forecast d = [60, 70, 50, 40, 80, 30] (thousands), S = $1,000, H = $0.005/unit/month.

- Start order in month 1.
- AC(1) = 1000 / 1 = $1,000.
- AC(2) = (1000 + 0.005 . 1 . 70,000) / 2 = (1000 + 350) / 2 = $675. Falling - extend.
- AC(3) = (1000 + 0.005 . (1 . 70k + 2 . 50k)) / 3 = (1000 + 850) / 3 = $617. Still falling - extend.
- AC(4) = (1000 + 0.005 . (70k + 100k + 120k)) / 4 = (1000 + 1,450) / 4 = $612. Marginally falling.
- AC(5) = (1000 + 0.005 . (70k + 100k + 120k + 320k)) / 5 = (1000 + 3,050) / 5 = $810. Rising - stop.

So the first order covers months 1-4 (total 220 k units). Then restart from month 5, repeat the rule, and continue. Result for Airtel: typically 2-3 orders per six-month window, with a total cost very close to Wagner-Whitin but a fraction of the compute. Silver-Meal is our default for Airtel-shaped series.

When to use which - decision rule of thumb

Demand shape	Algorithm	Reason
High-volume, smooth (GP, Banglalink, Robi)	EOQ	Constant-demand assumption holds; closed form is the cheapest.
Mid-volume, mildly varying (Airtel)	Silver-Meal	Near-optimal at O(T) cost - good practical balance.
Low-volume, lumpy or declining (Citycell, Teletalk)	Wagner-Whitin	Variable demand breaks EOQ; DP guarantees the optimum.

What this lets us decide

- **Order size per shipment** - fixed Q* for GP / Banglalink / Robi; variable Q_t for Citycell / Teletalk.
- **Order frequency** - D / Q* tells you whether to negotiate weekly, monthly or quarterly deliveries with the supplier.

- **Working-capital lock-up** - (Q* / 2) . unit_cost is the average inventory value sitting in the warehouse.
- **Supplier contract tiers** - knowing optimal cadence lets procurement target price breaks at that quantity.
- **Safety-stock layer** - the 51 IQR-flagged shocks from slide 16 calibrate SS = z . sigma . sqrt(L), added on top of Q*.
- **Service-level trade-off** - explicit 95% vs 98% target translates into the z-multiplier on safety stock.
- **Sensitivity analysis** - re-run Q* across $S \in [500, 5000]$ and $H \in [0.04, 0.10]$ to check robustness to cost assumptions.
- **Citycell exit signal** - $D \approx 0$ (and trending negative) means no procurement at all; lot-sizing tells you to wind the SKU down rather than just order less of it.

6. Future direction

Where the project goes after this deck.

Immediate next steps (Presentation 3)

- Train ARIMA per operator on the 96-month series; pick (p, d, q) via AIC.
- Train LSTM with a sliding window of recent months plus the engineered growth / share features.
- Train XGBoost with lag features, calendar dummies and the outlier flag.
- Blend into a stacked ensemble; evaluate with MAPE, RMSE, MAE on a holdout split.
- Run EOQ, Wagner-Whitin and Silver-Meal on the resulting demand vector and compare total cost across scenarios.

Beyond Presentation 3

- Bring in **exogenous signals** - GDP growth, FX, ARPU, BTRC regulatory news - to harden the long-horizon forecast.
- Extend the planning horizon from monthly to **weekly** once higher-frequency stock-keeping data is available.
- Add a **probabilistic forecast** (quantile LSTM / Bayesian ARIMA) so safety stock is calibrated to an explicit service level rather than a fixed z-score.
- Productionise as a **Streamlit / FastAPI dashboard** that re-runs the lot-sizing plan whenever the forecast updates.
- **Generalise**: swap the telecom subscriber series for any demand series (FMCG, retail, energy) - the same EDA + ensemble + lot-sizing pipeline transfers directly.

One-line takeaway

> *The EDA in this deck is not just data hygiene - it is the feature foundation that lets a single dataset drive both a demand forecast and a procurement plan in the same pipeline.*